SYSTEM DESIGN INTERVIEW

Design Movie Ticket Booking

BookMyShow / Fandango: venues, showtimes, and double-booking-proof seat reservation

~ 45 minute read

What's Inside

01 Problem Statement

Requirements and scope

Design a movie ticket booking platform like **BookMyShow** or **Fandango**. Users browse cities, theatres, movies, and showtimes; select specific seats on a screen layout; hold them during checkout; and pay. The hardest sub-problem is **preventing double-booking** when tens of thousands of users hammer a single show simultaneously — an Avengers premiere, a Coldplay concert, an IPL final.

Clarifying Questions

Question	Answer
Scope?	Movies first; concerts/sports later (same primitives)
Geography?	Multi-country; per-city catalog and currency
Seat selection?	Yes — visual seat map; not just count of tickets
Hold during checkout?	Yes — 5-minute hold while user pays
Payments?	Card, UPI, wallets via 3rd-party PSP (idempotent)
Refunds & cancellations?	Up to 2 hours before showtime; partial fee
Flash crowds?	Marvel premieres: 50K reservation attempts/sec on one show
Consistency?	Strong on seats; eventual on search/recommendations

02 Requirements

Functional and non-functional

Functional Requirements

Requirement	Details
Browse catalog	City → cinema → movie → showtime; filters (language, format)
Seat map	Render screen layout with AVAILABLE / HELD / BOOKED state
Hold seat	Place 5-minute hold; reject if seat already HELD/BOOKED
Checkout & pay	Idempotent charge via PSP; convert hold → booking on success
Cancel / refund	Pre-show cancellation; release inventory back to AVAILABLE
Notifications	Email/SMS/push for booking confirmation & reminders
Search	Full-text by movie title, genre, language, location

Non-Functional Requirements

Requirement	Target
Correctness	**Zero double-bookings** — invariant, not best-effort
Latency	<200ms seat map; <500ms hold; <3s end-to-end checkout
Availability	99.95% on booking path; 99.9% on search/discovery
Throughput	~1M bookings/day; 50K seat-reservations/sec on a hot show
Scalability	Multi-region; isolate hot shows so they don't starve cold ones
Fairness	Reasonable FIFO under contention; no permanent loser problem

03 Capacity Estimation

Math for scale

Users and Bookings

- Registered users: **10M**; DAU on weekends: ~2M
- Bookings per day: **1M** (avg 2.4 seats/booking ≈ 2.4M tickets/day)
- Average sustained bookings/sec: 1M / 86,400 ≈ **12 bookings/sec**
- Friday/weekend peak: **10×** sustained → ~120 bookings/sec average
- Read:write ratio (browse vs book): ~**50:1** → ~6K browse QPS sustained, 60K at peak

The Hot-Show Problem

- Marvel premiere goes on sale at 10:00 IST: tickets vanish in < 60 seconds
- One show = ~250 seats; 50K users hit *that single show* in the first second
- **Peak QPS on one show: 50K seat-reservation attempts/sec**
- Of those, only ~250 will succeed; the rest must fail fast and cleanly
- Implication: hot-show traffic is **1000× the average booking rate**; cannot be handled by the same path

Storage

- Cinemas: ~10K worldwide × ~10 screens each = **100K screens**
- Showtimes: 5 shows/screen/day × 100K × 365 = **180M showtimes/year**
- Seat inventory rows: 180M shows × ~200 seats avg = **36B rows/year**
- Booking record: ~500B; 1M/day × 365 = **~180 GB/year** bookings table
- Total transactional store with indexes & 5y retention: **~10–15 TB**

> KEY
>
> **Key Numbers**
>
> 10M users · 1M bookings/day · 10× Friday peak · **50K reservations/sec on a single hot show** · 100K screens · 36B seat rows/year · zero double-bookings tolerated.

04 Domain Model

Entities and relationships

Booking systems live or die on a clean domain model. The hierarchy is **chains** → **cities** → **cinemas** → **screens** → **seats**, with **movies** orthogonal to venues and connected through **showtimes**. **Bookings** are the leaf transactional record.

Entity Hierarchy

- **Chain** — operator (PVR, AMC, INOX); owns many cinemas
- **City** — geographic & timezone bucket; cinemas live in cities
- **Cinema** — physical venue; has 1..N screens
- **Screen** — auditorium; fixed seat layout (rows × cols, format: 2D, 3D, IMAX)
- **Seat** — physical seat in a screen; identified by (screen_id, row, col); has class (recliner, premium, standard)
- **Movie** — title, language(s), runtime, certification, posters
- **Showtime** — (screen_id, movie_id, start_time); the unit you sell
- **Seat Inventory** — per-(showtime, seat) row tracking AVAILABLE/HELD/BOOKED
- **Booking** — user_id + showtime_id + N seats + payment_id + status

Why Seat Inventory is Per-Showtime

- The same physical seat is sold many times across different showtimes
- State (AVAILABLE/HELD/BOOKED) is a property of *seat* × *showtime*, not seat alone
- This is the table that experiences flash-crowd contention, so it deserves its own design
- Pre-allocating one row per (showtime, seat) at showtime-creation simplifies UPDATE-only writes (no INSERT race)

> **INSIGHT**
>
> **Pre-allocate or Just-in-Time?**
>
> Pre-allocating seat_inventory rows when a showtime is created (~200 rows per show) lets the booking path do `UPDATE ... WHERE status='AVAILABLE'` with no insert path. JIT inserting on first hold avoids rows for never-booked shows but introduces an INSERT-vs-UPDATE race. **Pre-allocation wins**: 36B rows/year is cheap, contention-safety is priceless.

05 High-Level Architecture

System overview

The platform decomposes into a discovery side (Search, Catalog) optimized for read scale and an inventory side (Booking, Payment) optimized for correctness. A Notifications service is asynchronous off Kafka.

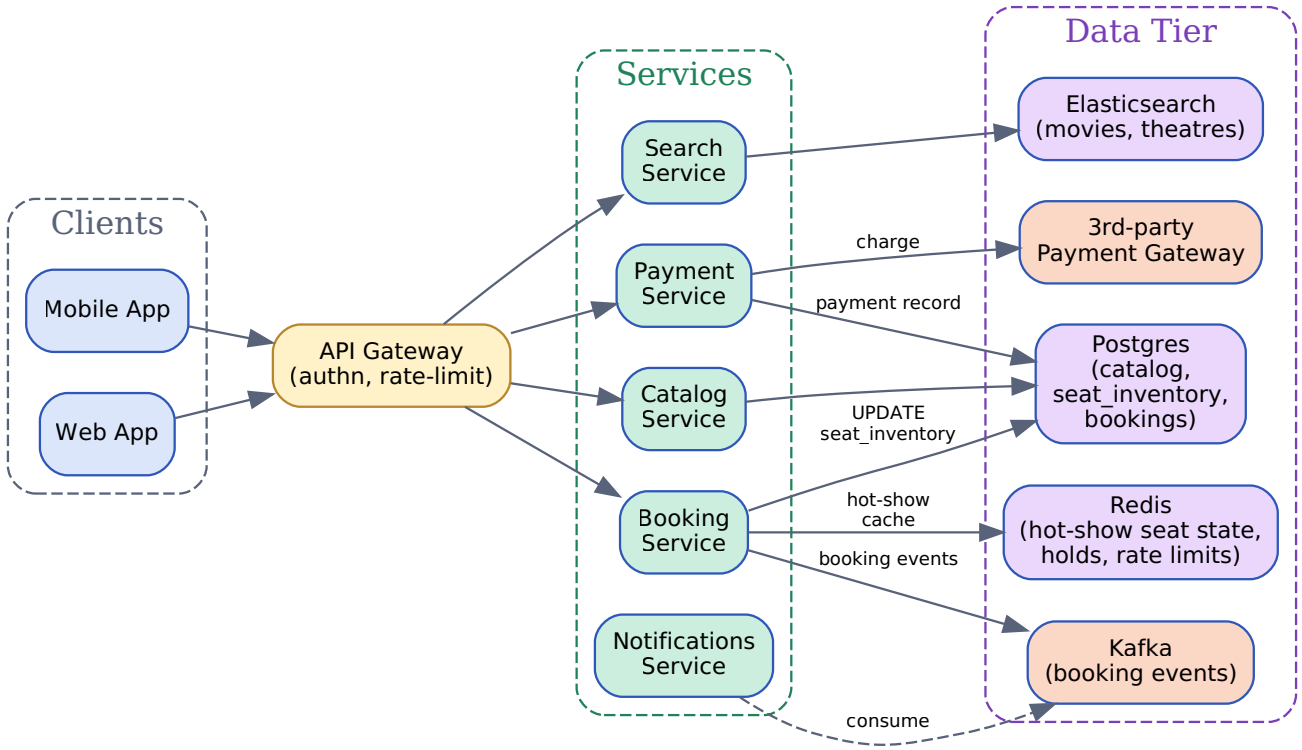

Figure 5.1: Mobile/Web → API Gateway fans out to Search (Elasticsearch), Catalog (Postgres), Booking (Postgres + Redis), Payment (PSP), Notifications. Kafka is the event spine.

Service Responsibilities

- **API Gateway:** authn (JWT), rate-limit per user/IP, route to services, terminate TLS
- **Search Service:** Elasticsearch query for movie/theatre/showtime; read-heavy, cacheable
- **Catalog Service:** CRUD for cinemas, screens, seats, movies, showtimes; partner-facing admin APIs
- **Booking Service:** seat hold + confirm; the correctness-critical path
- **Payment Service:** idempotent PSP integration; wraps the booking saga
- **Notifications Service:** Kafka consumer; sends email/SMS/push asynchronously

06 Database Schema

Tables for catalog, inventory, bookings

Showtime

```
CREATE TABLE showtimes (
  showtime_id    BIGINT PRIMARY KEY,
  screen_id      BIGINT NOT NULL REFERENCES screens(screen_id),
  movie_id       BIGINT NOT NULL REFERENCES movies(movie_id),
  start_time     TIMESTAMPTZ NOT NULL,
  end_time       TIMESTAMPTZ NOT NULL,
  language       VARCHAR(16) NOT NULL,
  format         VARCHAR(16) NOT NULL,   -- 2D, 3D, IMAX, 4DX
  base_price     INT          NOT NULL,  -- cents
  currency       CHAR(3)      NOT NULL,
  status         VARCHAR(16)  NOT NULL,  -- SCHEDULED, OPEN, SOLD_OUT, CANCELLED
  created_at     TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE (screen_id, start_time)
);
CREATE INDEX idx_show_movie_time ON showtimes (movie_id, start_time);
CREATE INDEX idx_show_screen_time ON showtimes (screen_id, start_time);
```

Seat Inventory (the contended table)

```
CREATE TABLE seat_inventory (
  showtime_id    BIGINT      NOT NULL,
  seat_id        BIGINT      NOT NULL,    -- (screen_id, row, col)
  status         VARCHAR(16) NOT NULL,    -- AVAILABLE, HELD, BOOKED
  hold_id        UUID        NULL,        -- set when HELD
  user_id        BIGINT      NULL,        -- holder/buyer
  expires_at     TIMESTAMPTZ NULL,        -- hold expiry
  version        INT         NOT NULL DEFAULT 0,  -- optimistic lock
  price          INT         NOT NULL,    -- frozen at allocation
  PRIMARY KEY (showtime_id, seat_id)
);
CREATE INDEX idx_holds_expiry
  ON seat_inventory (expires_at)
  WHERE status = 'HELD';
```

Booking

```
CREATE TABLE bookings (
  booking_id     UUID         PRIMARY KEY,
  user_id        BIGINT       NOT NULL,
  showtime_id    BIGINT       NOT NULL,
  seat_ids       BIGINT[]     NOT NULL,
  hold_id        UUID         NOT NULL,
  amount         INT          NOT NULL,
  currency       CHAR(3)      NOT NULL,
  status         VARCHAR(16)  NOT NULL,  -- PENDING, CONFIRMED, FAILED, CANCELLED
  payment_id     UUID         NULL,
  idempotency_key VARCHAR(64) NOT NULL UNIQUE,
  created_at     TIMESTAMPTZ  DEFAULT NOW(),
  confirmed_at   TIMESTAMPTZ  NULL
);
CREATE INDEX idx_bk_user ON bookings (user_id, created_at DESC);
CREATE INDEX idx_bk_showtime ON bookings (showtime_id);
```

TIP

Why a version column on seat_inventory

The `version` column enables **optimistic locking**: a hold attempt reads (status, version) and then writes only if version still matches. Under contention, losers retry instead of blocking — far better than row-level lock queues at 50K QPS.

07 Seat Reservation: Three Approaches

Pessimistic vs Optimistic vs Distributed Lock

The single most important interview question for this design: **how do you guarantee that a seat is booked by exactly one user?** There are three industry-standard primitives, each with different consistency, latency, and operational characteristics.

(a) Pessimistic — SELECT FOR UPDATE per seat

```
BEGIN;
SELECT status FROM seat_inventory
 WHERE showtime_id = $1 AND seat_id = ANY($2)
 FOR UPDATE;                          -- row lock until COMMIT
-- if any row is not AVAILABLE: ROLLBACK and fail
UPDATE seat_inventory
   SET status='HELD', hold_id=$3, user_id=$4, expires_at=NOW()+INTERVAL '5 min'
 WHERE showtime_id=$1 AND seat_id=ANY($2);
COMMIT;
```

- **Pros:** bulletproof correctness; serializes contenders cleanly
- **Cons:** row locks held for the duration of the txn; on a hot show, 50K writers serialize through Postgres → seconds-long queues, connection exhaustion
- **When to use:** normal-traffic shows; *and* as the inner safety net inside the checkout transaction even on hot shows

(b) Optimistic — version column + conditional UPDATE

```
-- read current state (may be served from a replica)
SELECT status, version FROM seat_inventory
 WHERE showtime_id=$1 AND seat_id=$2;

-- write only if no one has touched the row since
UPDATE seat_inventory
   SET status='HELD', hold_id=$3, user_id=$4,
       expires_at=NOW()+INTERVAL '5 min',
       version = version + 1
 WHERE showtime_id=$1 AND seat_id=$2
   AND status='AVAILABLE'
   AND version=$5;
-- rowcount = 1 → won; 0 → lost, retry or pick another seat
```

- **Pros:** no locks held across the request; loser sees rowcount=0 and retries; scales horizontally

- **Cons:** requires explicit retry handling and a UX path ("this seat was just taken — pick another"); more complex client
- **When to use:** the **default** for everyday traffic — most hold attempts will be uncontended single-row writes

(c) Distributed Lock in Redis (RedLock)

```
# pseudocode: try to acquire a per-seat lock for ~6s
key   = f"seat:{showtime_id}:{seat_id}"
token = uuid4().hex
ok    = redis.set(key, token, nx=True, px=6000)   # SET NX PX
if not ok:
    return SeatTaken
try:
    # inside the lock: do the DB hold (still verify status!)
    rowcount = db.execute(OPTIMISTIC_HOLD_SQL, ...)
    if rowcount == 0: return SeatTaken
finally:
    # release only if we still own the lock (Lua CAS)
    redis.eval(RELEASE_LUA, keys=[key], args=[token])
```

- **Pros:** sub-millisecond lock acquisition; absorbs the 50K-QPS thundering herd before it hits Postgres; trivial to put in front of any DB
- **Cons:** RedLock has well-known partition-tolerance edge cases (Kleppmann's critique); a Redis failover can briefly grant the same lock twice
- **Mitigation:** always do the DB conditional UPDATE inside the lock — Redis is the funnel, Postgres is the source of truth

Comparison

Aspect	Pessimistic (SELECT FOR UPDATE)	Optimistic (version)	Redis Distributed Lock
Correctness	Strongest (DB serializes)	Strong (CAS in DB)	Strong only if DB still validates
Latency under contention	High (lock waits)	Low (fail-fast retry)	Lowest (fail at Redis)
Throughput on hot row	Bottlenecked by lock queue	Many losers, but all fast	Funnels traffic; DB sees 1 winner
Failure modes	Long-running txn → deadlock	Retry storms if naive	RedLock partition edge cases

Aspect	Pessimistic (SELECT FOR UPDATE)	Optimistic (version)	Redis Distributed Lock
Complexity	Lowest	Medium (retry logic)	Highest (lock + CAS + lease)
Best for	Inside checkout txn always	Default everyday hold path	Hot-show admission control

KEY

Recommended Composition

Use **(b) optimistic locking** as the default seat-hold primitive — it scales linearly and fails fast. For known hot shows, put a **(c) Redis lock** in front to absorb the thundering herd. Inside the eventual checkout transaction, always upgrade to **(a) SELECT FOR UPDATE** on the held rows so the final commit is bulletproof. All three layers are correct; the composition gives correctness *and* throughput.

08 Hold State Machine

AVAILABLE → HELD → BOOKED, with TTL

Status Transitions

- **AVAILABLE → HELD:** on successful hold; sets hold_id, user_id, expires_at = now + 5 min
- **HELD → BOOKED:** on payment success; clears expires_at, links to booking_id
- **HELD → AVAILABLE:** on payment failure, user cancel, or expiry sweep
- **BOOKED → AVAILABLE:** only on cancellation/refund; logged for audit
- **Invariant:** a row can be transitioned only by the owner of the current hold_id (or admin/sweeper)

Expiring Holds

If a user opens checkout and walks away, the seats must be released. Two mechanisms work in concert:

- **Background sweeper** — every 10 sec: `UPDATE seat_inventory SET status='AVAILABLE', hold_id=NULL, expires_at=NULL WHERE status='HELD' AND expires_at < NOW()`; uses the partial index
- **Lazy release on read** — when seat map is fetched, treat HELD-with-expired-expires_at as AVAILABLE in the API response (UX fast path), then enqueue the sweep
- **Per-hold TTL in Redis** — for hot shows, a Redis key `hold:{hold_id}` with PX=300s; on expiry, a key-expiry listener kicks the DB sweep

Optimistic Hold with Retry (algorithm)

```
def hold_seats(showtime_id, seat_ids, user_id, max_retries=3):
    hold_id = uuid4()
    expires = now() + timedelta(minutes=5)
    for attempt in range(max_retries):
        # 1. read current versions (one round-trip)
        rows = db.fetchall('''
            SELECT seat_id, status, version FROM seat_inventory
             WHERE showtime_id=%s AND seat_id = ANY(%s)
        ''', (showtime_id, seat_ids))
        if any(r.status != 'AVAILABLE' for r in rows):
            return Failure('SEAT_TAKEN', conflicting=[r.seat_id
                                    for r in rows if r.status != 'AVAILABLE'])

        # 2. CAS update — all-or-nothing in one txn
        with db.transaction():
            updated = 0
            for r in rows:
```

```
                updated += db.execute('''
                    UPDATE seat_inventory
                       SET status='HELD', hold_id=%s, user_id=%s,
                           expires_at=%s, version=version+1
                     WHERE showtime_id=%s AND seat_id=%s
                       AND status='AVAILABLE' AND version=%s
                ''', (hold_id, user_id, expires,
                      showtime_id, r.seat_id, r.version)).rowcount
            if updated == len(rows):
                return Success(hold_id, expires)
            db.rollback()  # partial → all-or-nothing
        # 3. someone moved a row; loop and re-read
    return Failure('SEAT_TAKEN_RETRIES_EXHAUSTED')
```

WARNING

All-or-Nothing Group Holds

A user picking 4 seats expects **all 4 or none**. Run all per-seat CAS updates inside a single transaction; if any one fails, rollback the whole transaction and retry. Never confirm a partial group — that's a UX disaster ("you got 3 of 4").

09 Seat Reservation Sequence

End-to-end flow, click to confirmation

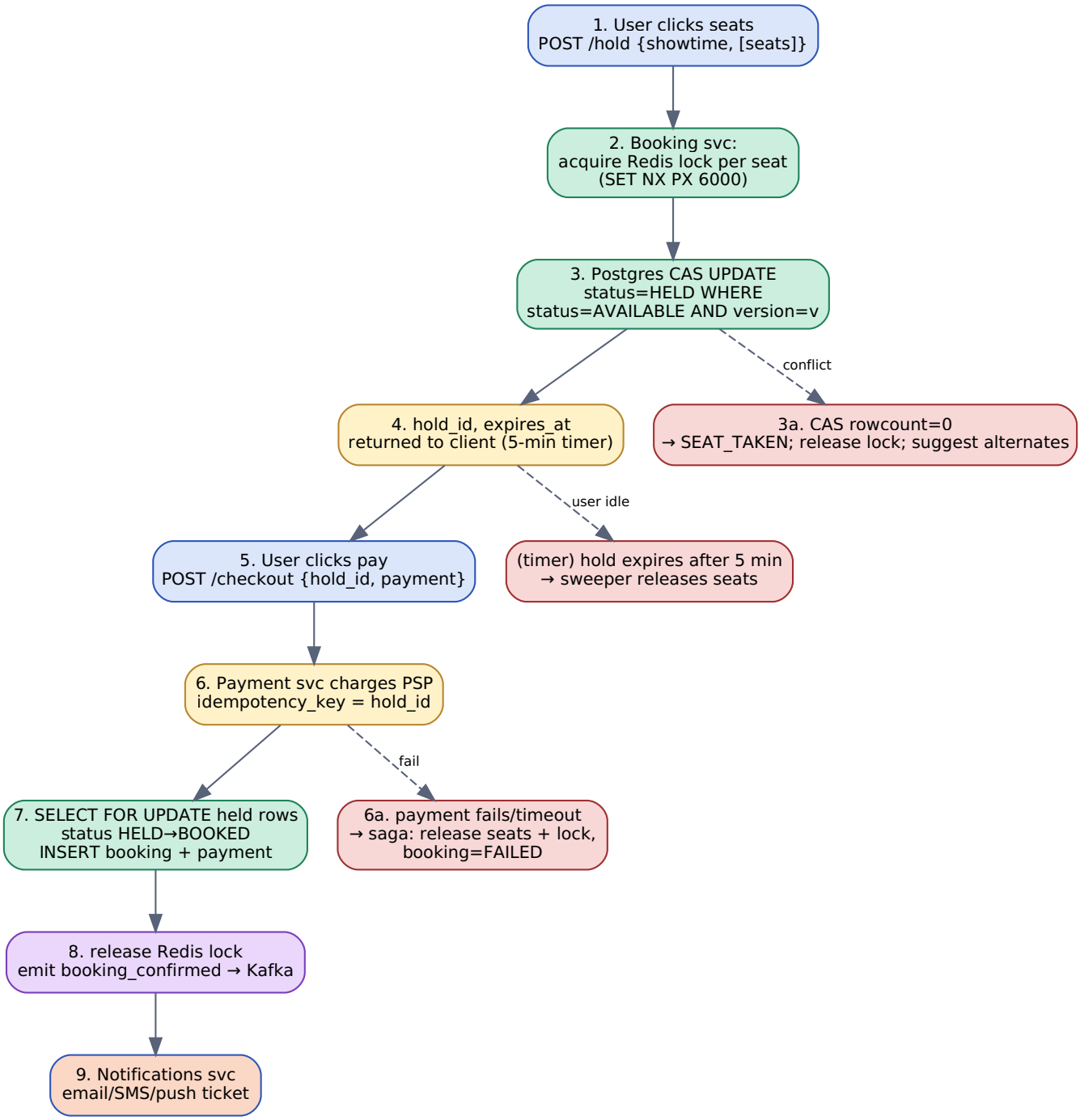

Figure 9.1: User clicks seats → Booking Service tries Redis admission lock → Postgres optimistic CAS hold → checkout → idempotent payment → confirm booking → release Redis lock. Failures at any step trigger compensating actions.

Step-by-Step

1. **Hold attempt:** client sends seat ids; Booking acquires per-seat Redis locks (admission control)
2. **DB CAS:** within the locks, run optimistic UPDATE; require all rows updated or rollback
3. **Hold confirmed:** return `hold_id` + `expires_at`; client starts a 5-min countdown
4. **Checkout:** client posts payment details with `idempotency_key = hold_id`
5. **PSP charge:** Payment Service issues idempotent charge; retries on network errors are safe
6. **Confirm:** on success, transaction does SELECT FOR UPDATE on the HELD rows, transitions to BOOKED, inserts booking + payment record
7. **Release lock & emit:** drop Redis lock; publish `booking_confirmed` to Kafka
8. **Notify:** Notifications consumer emails/SMSs the ticket; Search service may evict cached seat-map

10 Payment Integration

Idempotency and the seat-release saga

Payment is where money meets seats; it's also the noisiest part of the system (PSP timeouts, network drops, user back-button). The two design goals are **idempotency** (so retries don't double-charge) and a **compensating transaction** (so a failed payment cleanly returns seats to inventory).

Idempotent Charge

- **Idempotency key:** use `hold_id` (UUID) as the key sent to the PSP
- **PSP guarantee:** Stripe/Razorpay/Adyen dedupe by key for 24h; same key returns the same charge result
- **Local table:** `payments(payment_id, idempotency_key UNIQUE, status, psp_ref, ...)`; INSERT-or-fetch on the unique key
- **Client retries:** always allowed; the same hold_id always converges to the same booking

Compensating Saga on Failure

```
def checkout(hold_id, payment_method):
    booking = create_booking_pending(hold_id)         # status=PENDING
    try:
        result = psp.charge(amount, payment_method,
                            idempotency_key=hold_id,
                            timeout=20s)
    except (PspTimeout, PspNetworkError):
        # don't know yet — schedule reconciler
        kafka.emit('payment_unknown', booking.id, hold_id)
        return Pending(booking.id)
    if result.status == 'SUCCESS':
        confirm_booking(booking.id, hold_id, result.payment_id)
        return Confirmed(booking.id)
    else:
        # COMPENSATING TRANSACTION: release the seats
        release_hold(hold_id)
        mark_booking_failed(booking.id, result.error)
        return Failed(result.error)
```

Reconciler for Unknown States

- PSP timeouts leave the charge in a **UNKNOWN** state — could be charged or not
- Reconciler job polls the PSP every 30s on the idempotency_key until terminal

- On terminal SUCCESS: confirm booking; on FAIL: release hold (idempotent if already released)
- Hold TTL (5 min) is the safety net: even if reconciler is down, seats free themselves

> **WARNING**
>
> **Never confirm before charge succeeds**
>
> Tempting optimization: "emit booking_confirmed eagerly to keep the UI snappy." Don't. If the charge later fails, you've already sent an email with a ticket QR code that doesn't exist — and the seat is double-bookable. **Confirmation must follow successful capture.**

11 Search & Discovery

Finding the right show

Query Surface

- **By location:** city → list of cinemas with shows today; default user-detected city
- **By movie:** showtimes across cinemas in user's city, grouped by venue
- **By language / format:** Hindi, Tamil, …; 2D, 3D, IMAX, 4DX
- **By time window:** tonight, weekend, this week
- **Full-text:** movie title, cast, theatre name (typo-tolerant)

Storage & Indexing

- **Postgres** is source of truth for catalog (movies, theatres, showtimes)
- **Elasticsearch** is the read-side projection for search; indexed via CDC (Debezium → Kafka → ES sink)
- **Per-city sharding** in ES: index alias `showtimes-{city}`; query routes by user city
- **TTL on documents:** showtimes in the past auto-delete from ES (ILM policy, 7-day retention)

Caching Discovery

- **Edge cache (CDN):** movie list per city, posters, trailers — TTL 5 min
- **App cache (Redis):** showtimes-by-(city,date) keyed lookups, TTL 60s
- **Seat map snapshot:** AVAILABLE/HELD/BOOKED state cached for 2s — fresh enough for browsing, but never trust for booking
- **Stale-while-revalidate:** serve last-good while async refresh runs

> **INSIGHT**
>
> **Read state is advisory; write state is authoritative**
>
> Seat maps shown during browsing can be a few seconds stale — that's fine, users still pick a seat. The *hold attempt* is the only place where freshness matters, and it always re-reads from the DB. Decoupling display freshness from correctness freshness is the unlock for browsing throughput.

12 Hot-Show Special Path

Surviving 50K reservations/sec on one show

An Avengers premiere on sale at 10:00 IST is a fundamentally different workload from the average booking. We pre-classify shows as **hot** and route them through a dedicated path.

Identifying Hot Shows

- **Manual flag** — partner ops mark known blockbuster premieres as hot at showtime creation
- **Auto-detect** — if seat-map QPS for a showtime crosses, e.g., 1K/sec, promote to hot
- **Hot path activates:** Redis-fronted admission, dedicated Postgres connection pool, separate rate limiter, queueing UI

Admission Control

- **Virtual waiting room:** users entering the show page join a token queue (Redis sorted set); only N=2× capacity admitted concurrently
- **Seat-level locks:** Redis SET NX PX serializes contenders for each seat — only one reaches Postgres per seat
- **Per-user rate limit:** max 1 outstanding hold attempt per user per show (prevents bot floods)
- **Connection isolation:** hot-show booking pool ≠ general pool, so the rest of the system stays healthy

Caching Seat State for Hot Shows

- **Per-seat status cached in Redis:** hash `show:{id}:seats`, fields = seat_id → status
- **Updated on every transition:** Booking writes both DB and Redis; on conflict, DB wins (CDC repairs cache)
- **Seat-map reads bypass Postgres entirely** during hot windows → 99% browse load on Redis
- **Fallback:** if Redis is unhealthy, seat map falls back to DB (slower, but correct); booking still works because DB is source of truth

> KEY
>
> **Three-Layer Funnel**
>
> 1) **Waiting room** rate-limits how many users can attempt holds at all. 2) **Redis lock** serializes the survivors per seat. 3) **Postgres CAS** is the final source-of-truth gate. Each layer drops ~10–100× of the load. The 50K/sec storm becomes ~250 successful writes/sec at the DB — well within normal capacity.

13 Failure Modes & Recovery

What breaks and how we degrade

Failure	Impact	Detection	Mitigation
Payment timeout after hold	Money may or may not be charged	PSP HTTP timeout (>20s)	Reconciler polls PSP on idempotency_key; release saga frees seats; hold TTL is safety net
Redis cluster down	Hot-show admission lost; cache miss storm	Redis health check, latency alarm	Fall back to DB-only path; degrade hot-show throughput; reject new attempts at 429
Postgres primary down	All booking writes fail	Conn errors, replication lag alarm	Auto-promote replica (Patroni); booking returns 503 with retry-after; reads served from replica
Long-running SELECT FOR UPDATE	Lock queue, conn exhaustion	pg_locks monitoring	Statement_timeout=2s; circuit breaker; switch hot show to optimistic+Redis path
Sweeper lag	Expired holds linger; seats look unavailable	Holds older than expires_at metric	Multi-instance sweeper; lazy release on read; alert if backlog > 1 min
Kafka down	Notifications delayed; analytics lag	Broker dead alert	Booking core path still works; events buffered locally and replayed
PSP webhook lost	Booking stuck PENDING	Booking-aging metric	Reconciler polls; webhook retry; user-facing "check status" endpoint
Showtime cancelled by cinema	Bookings invalid	Partner admin event	Bulk refund saga; emit notifications; release inventory

Degradation Order

- **Tier 1 (must work):** seat hold + payment + booking confirmation
- **Tier 2 (degrade gracefully):** hot-show waiting room, seat-map cache, search relevance
- **Tier 3 (delay OK):** notifications, analytics, recommendations

> **WARNING**
>
> **Saga, not 2PC**

> Payment + Inventory live in different systems (PSP + Postgres). 2PC across them is neither possible nor desirable — the PSP is a 3rd party. Use a saga: forward steps (create booking → charge → confirm) each have a compensating reverse (release hold → void/refund). Every state transition is durable and replayable.

14 Design Trade-offs

Decisions and rationale

Decision	Choice	Trade-off
Concurrency primitive (default)	Optimistic locking (version)	Fast under low contention; needs retry UX. Pessimistic would queue everyone and starve at 50K QPS.
Concurrency primitive (hot show)	Redis admission + DB CAS + checkout SELECT FOR UPDATE	Three layers add complexity; without them a single hot show DDoSes the booking DB.
Seat inventory rows	Pre-allocate at showtime creation	36B rows/yr storage; eliminates INSERT race and allows pure UPDATE-only path.
Source of truth	Postgres	ACID gives us correctness; sharding & ops overhead. NoSQL would be cheaper to scale but weaker on multi-row transactional holds.
Hold TTL	5 minutes	Long enough for users to pay; short enough that abandoned holds free quickly. Shorter = friction; longer = inventory hoarding.
Search store	Elasticsearch projection from Postgres	Eventual consistency on browse data is OK; ES lets us scale read QPS independently of catalog writes.
Payment confirmation	Synchronous within checkout txn	Slower checkout; never overbooks. Async confirmation would be faster but lets you ship a ticket whose payment later fails.
Notifications	Async via Kafka	User sees confirmation page instantly; email may take a few seconds. Sync would couple booking latency to email infra.
Hot-show detection	Hybrid (manual flag + auto-detect)	Manual catches premieres; auto catches surprises. Pure auto reacts late; pure manual misses unexpected hits.

15 Scalability & Multi-Region

Growing the platform

Horizontal Scaling

- **Stateless services** — Booking, Search, Payment scale behind load balancers
- **Postgres sharded by city_id** — bookings rarely cross cities; 32–128 shards depending on country
- **Read replicas per shard** — serve seat-map browsing and analytics; primary handles holds/bookings
- **Redis cluster** — slot-based; one slot per show key so all that show's locks colocate

Sharding by City

- **Why city:** a booking is always against one cinema in one city → no cross-shard txns on the hot path
- **Hot-city overflow:** Mumbai is bigger than 10× a small city — sub-shard popular cities by cinema_id
- **Catalog reference data** — movies, languages — globally replicated read-only tables

Multi-Region

- **Region per country** — IN, US, EU isolated; latency and data-residency benefits
- **No cross-region writes on hot path** — a Mumbai booking never touches the EU primary
- **Global services:** identity (users), payment routing, fraud, recommendations — replicated read views
- **Disaster recovery:** per-region warm standby; RPO < 1 min via streaming replication

> **INSIGHT**
>
> **Why City is the Natural Shard Key**
>
> Movie ticket booking is fundamentally local: users pick a cinema in their city, showtimes belong to one screen in one venue, holds touch one row per (showtime, seat). There is no realistic transaction that spans cities. That makes city_id the right shard key, gives us geo-locality for free, and isolates hot-city blast radius.

16 Interview Playbook

How to present this

Movie ticket booking is the canonical **contended-inventory** interview question. Hotel rooms, flight seats, concert tickets, restaurant reservations all share the same core. If you can defend the seat-reservation design under flash-crowd pressure, you have answered 80% of the question.

45-Minute Interview Outline

1. **Intro (3 min):** 10M users, 1M bookings/day, 10× Friday peak, **50K/sec on one hot show**; zero double-bookings
2. **Domain model (5 min):** chains → cinemas → screens → seats → showtimes → seat_inventory → bookings; pre-allocate inventory rows
3. **High-level arch (3 min):** API GW → {Search, Catalog, Booking, Payment, Notifications} → {Postgres, Redis, ES, Kafka, PSP}
4. **Seat reservation (15 min — the centerpiece):** walk through (a) pessimistic, (b) optimistic, (c) Redis lock; recommend (b) default, (a) inside checkout txn, (c) for hot shows
5. **State machine + saga (5 min):** AVAILABLE → HELD → BOOKED; 5-min TTL; sweeper; payment failure → release saga
6. **Hot-show path (5 min):** waiting room + Redis admission + connection isolation
7. **Failures (5 min):** payment timeout, Redis down, sweeper lag, PSP webhook loss
8. **Trade-offs & wrap (4 min):** consistency vs throughput, pre-allocate vs JIT, sync confirm vs async

Key Talking Points

- "50K reservation attempts/sec on one show" — frames the entire concurrency discussion
- "AVAILABLE → HELD → BOOKED with hold_id and expires_at" — concrete state machine
- "Optimistic CAS with version column" — default; "SELECT FOR UPDATE inside checkout txn" — final safety net
- "Redis lock funnels 50K/sec into 250 DB writes" — quantifies the hot-show admission
- "hold_id as PSP idempotency key" — payment retries cannot double-charge or double-book
- "Compensating saga, not 2PC" — release seats on payment failure
- "Pre-allocate seat_inventory rows" — UPDATE-only path; no INSERT race
- "City as shard key" — bookings never cross cities; clean horizontal scale

Common Follow-ups & Answers

- **Q: How do you guarantee no double-booking?** A: Postgres UPDATE … WHERE status='AVAILABLE' AND version=v is atomic; rowcount=1 means we won, 0 means lost. Inside the checkout txn, SELECT

FOR UPDATE the held rows for an extra serialization barrier. Redis is only a funnel, never the source of truth.

- **Q: What if payment succeeds but booking confirm fails?** A: Retry the confirm; idempotent on hold_id. Reconciler reads payment status; if charged but no booking after N retries, refund and release seats. Hold TTL is the backstop.
- **Q: What if 50K users want the same seat?** A: Redis SET NX serializes them; only one reaches the DB; the other 49,999 see SEAT_TAKEN within milliseconds. UI suggests next-best alternates.
- **Q: Why not just SELECT FOR UPDATE everywhere?** A: At 50K QPS the lock queue and connection count blow up Postgres. Optimistic + Redis admission keeps lock duration to a single fast UPDATE.
- **Q: How do you handle the user closing the tab?** A: 5-min TTL; background sweeper releases; lazy release on read returns seat as AVAILABLE if expires_at < now.
- **Q: Refunds and cancellations?** A: Cancellation flips BOOKED → AVAILABLE in the same txn that issues the refund; emit cancellation event; partial-fee policy lives in the booking rules engine.
- **Q: How do you scale beyond one DB?** A: Shard by city_id; bookings are local-by-construction. Hot cities sub-shard by cinema_id.

KEY

Memorize These Numbers

10M users · 1M bookings/day · 10× Friday peak · **50K reservation attempts/sec on a hot show** · 5-min hold TTL · ~250 seats/show · 100K screens · AVAILABLE → HELD → BOOKED · zero double-bookings.